

React JS Notes — Props, Component Communication, Events, Conditional Rendering, Lists & Forms

These notes are designed for **beginners and interns**. We will build one small React application step-by-step so students understand **theory + code + CSS + implementation + assignments** together.

We will use **React + Vite**.

1. What We Will Learn

By the end of this lesson, students will understand:

Props & Component Communication

- Props
- Multiple props
- Destructuring
- Parent → Child communication
- Reusable components
- Component composition

Events & Conditional Rendering

- Event handling
- Click events
- Input events
- Event parameters
- Conditional rendering
- Ternary operator ? :
- Logical &&

Lists & Forms

- map()
 - Keys
 - Rendering objects
 - Controlled inputs
 - Forms
 - Select
 - Checkbox
 - Radio buttons
-

PART 1 — Understanding Components

2. What is a Component?

A React component is a reusable piece of UI.

For example, imagine a website:

App

└─ Header

└─ StudentCard

```
├── StudentCard
├── StudentCard
└── Footer
```

Instead of writing the same student-card HTML three times, we create one component:

`<StudentCard />`

and reuse it.

3. Create React Project

Open VS Code terminal.

Run:

```
npm create vite@latest react-props-events -- --template react
```

Go inside the project:

```
cd react-props-events
```

Install dependencies:

```
npm install
```

Start the application:

```
npm run dev
```

Open the URL shown by Vite, usually:

```
http://localhost:5173
```

4. Recommended Folder Structure

Create this structure:

```
react-props-events/
```

```
|
```

```
├── src/
```

```
| |
```

```
| | ├── components/
```

```
| | |   ├── Header.jsx
```

```
| | |   ├── StudentCard.jsx
```

```
| | |   ├── StudentList.jsx
```

```
| | |   └── StudentForm.jsx
```

```
| |
```

```
| | ├── App.jsx
```

```
| | ├── App.css
```

```
| | └── main.jsx
```

```
|
```

```
├── public/
```

```
|
```

```
├── package.json
```

```
└── index.html
```

PART 2 — Props

5. What are Props?

Props means **Properties**.

Props are used to pass data from a **parent component to a child component**.

Think of props like function arguments.

Normal JavaScript:

```
function greet(name) {  
  console.log(name);  
}
```

```
greet("Amit");
```

React:

```
<StudentCard name="Amit" />
```

The parent sends:

name = Amit

The child receives it through props.

6. Basic Props Example

App.jsx

```
import "./App.css";
```

```
function StudentCard(props) {  
  return (  
    <div className="card">  
      <h2>{props.name}</h2>  
      <p>Marks: {props.marks}</p>  
    </div>  
  );  
}
```

```
function App() {  
  return (  
    <div className="app">  
      <h1>Student Management</h1>  
  
      <StudentCard  
        name="Amit"  
        marks={85}  
      />  
    </div>  
  );  
}
```

```
<StudentCard
  name="Rahul"
  marks={92}
/>
</div>
);
}
```

export default App;

Here:

```
<StudentCard name="Amit" marks={85} />
```

passes two props:

name → Amit

marks → 85

7. Multiple Props

We can pass many props.

```
<StudentCard
  name="Amit"
  marks={85}
  city="Delhi"
  course="Computer Science"
  age={22}
/>
```

Child:

```
function StudentCard(props) {
  return (
    <div>
      <h2>{props.name}</h2>
      <p>Marks: {props.marks}</p>
      <p>City: {props.city}</p>
      <p>Course: {props.course}</p>
      <p>Age: {props.age}</p>
    </div>
  );
}
```

8. Props with Destructuring

Instead of:

props.name

props.marks

```
props.city
we can destructure props.
function StudentCard({ name, marks, city }) {
  return (
    <div className="card">
      <h2>{name}</h2>
      <p>Marks: {marks}</p>
      <p>City: {city}</p>
    </div>
  );
}
```

This is cleaner.

9. Props Are Read-Only

A child should not directly modify its props.

For example, avoid:

```
function StudentCard({ marks }) {

  marks = 100; // ❌ Don't do this

}
```

Props are controlled by the parent.

Think:

Parent

↓ props

Child

PART 3 — Reusable Components

10. Why Reusable Components?

Suppose we have:

Amit

Rahul

Priya

Neha

Instead of creating:

```
<div>...</div>
```

```
<div>...</div>
```

```
<div>...</div>
```

```
<div>...</div>
```

we create:

```
<StudentCard />
```

and reuse it.

11. Create StudentCard Component

src/components/StudentCard.jsx

```
function StudentCard({ name, marks, city, course }) {  
  return (  
    <div className="student-card">  
  
      <div className="student-avatar">  
        {name.charAt(0)}  
      </div>  
  
      <div className="student-info">  
        <h2>{name}</h2>  
  
        <p>  
          <strong>Course:</strong> {course}  
        </p>  
  
        <p>  
          <strong>City:</strong> {city}  
        </p>  
  
        <p>  
          <strong>Marks:</strong> {marks}  
        </p>  
      </div>  
  
    </div>  
  );  
}
```

```
export default StudentCard;
```

12. Use the Component

App.jsx

```
import './App.css';  
import StudentCard from './components/StudentCard';  
  
function App() {
```

```
return (  
  <div className="app">  
  
    <h1>Student Management System</h1>  
  
    <div className="student-container">  
  
      <StudentCard  
        name="Amit"  
        marks={85}  
        city="Delhi"  
        course="Computer Science"  
      />  
  
      <StudentCard  
        name="Rahul"  
        marks={92}  
        city="Mumbai"  
        course="Information Technology"  
      />  
  
      <StudentCard  
        name="Priya"  
        marks={78}  
        city="Pune"  
        course="Computer Science"  
      />  
  
    </div>  
  
  </div>  
);  
}  
  
export default App;
```

13. CSS

App.css

```
* {  
  margin: 0;  
  padding: 0;
```

```
    box-sizing: border-box;
}
```

```
body {
    font-family: Arial, sans-serif;
    background: #f4f7fb;
    color: #222;
}
```

```
.app {
    min-height: 100vh;
    padding: 40px;
}
```

```
.app h1 {
    text-align: center;
    margin-bottom: 30px;
    color: #1e293b;
}
```

```
.student-container {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
    gap: 20px;
    max-width: 1000px;
    margin: auto;
}
```

```
.student-card {
    background: white;
    padding: 25px;
    border-radius: 15px;
    display: flex;
    gap: 20px;
    align-items: center;
    box-shadow: 0 8px 20px rgba(0, 0, 0, 0.08);
    transition: 0.3s;
}
```

```
.student-card:hover {
    transform: translateY(-5px);
}
```

```

}

.student-avatar {
  width: 60px;
  height: 60px;
  border-radius: 50%;
  background: #4f46e5;
  color: white;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 25px;
  font-weight: bold;
}

.student-info h2 {
  margin-bottom: 8px;
}

.student-info p {
  margin: 5px 0;
  color: #64748b;
}

```

PART 4 — Component Composition

14. What is Component Composition?

Component composition means **building a larger component by putting smaller components inside it.**

Example:

App

↓

Dashboard

↓

StudentCard

For example:

```

function Dashboard() {
  return (
    <div>
      <Header />
      <StudentCard />
      <Footer />
    </div>
  )
}

```

```
    </div>
  );
}
```

This makes applications easier to maintain.

15. children Prop

React provides a special prop called:
children

Example:

```
function Card({ children }) {
```

```
  return (
    <div className="box">
      {children}
    </div>
  );
```

```
}
```

Use:

```
<Card>
```

```
  <h2>Hello Student</h2>
```

```
  <p>Welcome to React.</p>
```

```
</Card>
```

Everything between:

```
<Card>
```

```
...
```

```
</Card>
```

is available through children.

PART 5 — Events

16. What is Event Handling?

Events allow React applications to respond to user actions.

Examples:

Click

Typing

Submit

Mouse movement

Change

Focus

17. Click Event

```
function App() {  
  
  function handleClick() {  
    alert("Button clicked!");  
  }  
  
  return (  
    <button onClick={handleClick}>  
      Click Me  
    </button>  
  );  
}
```

Notice:

onClick={handleClick}

not:

onClick={handleClick()}

The second one calls the function immediately during rendering.

18. Inline Event

You can also write:

```
<button onClick={() => alert("Hello")}>  
  Click  
</button>
```

For larger logic, a separate function is generally easier to read.

19. Event Parameter

React passes an event object to the handler.

```
function handleClick(event) {  
  console.log(event);  
}
```

```
<button onClick={handleClick}>  
  Click  
</button>
```

For input:

```
function handleChange(event) {  
  console.log(event.target.value);  
}
```

```
<input onChange={handleChange} />
```

Important:

event.target.value
gets the current input value.

PART 6 — Conditional Rendering

20. What is Conditional Rendering?

Conditional rendering means:

Show different UI depending on a condition.

Example:

marks >= 40

↓

Pass

marks < 40

↓

Fail

21. Ternary Operator

Syntax:

condition ? trueValue : falseValue

Example:

```
function Result({ marks }) {
```

```
  return (  
    <div>  
      {marks >= 40 ? (  
        <p>Pass</p>  
      ) : (  
        <p>Fail</p>  
      )}  
    </div>  
  );
```

```
}
```

22. Ternary with CSS Class

```
<p className={marks >= 40 ? "pass" : "fail"}>  
  {marks >= 40 ? "Pass" : "Fail"}  
</p>
```

CSS:

```
.pass {  
  color: green;
```

```
font-weight: bold;
}
```

```
.fail {
  color: red;
  font-weight: bold;
}
```

23. Logical AND &&

Use && when you only want to show something if a condition is true.

```
{marks >= 90 && (
  <p>Excellent Performance!</p>
)}
```

If:

marks = 95

it shows the message.

If:

marks = 70

it doesn't show it.

PART 7 — Lists

24. Why Lists?

Real applications have multiple records.

Example:

Students:

Amit

Rahul

Priya

Neha

We shouldn't manually create a component for every student.

We can use:

map()

25. JavaScript map()

Example:

```
const numbers = [10, 20, 30];
```

```
numbers.map((number) => {
  return number * 2;
});
```

Result:

20
40
60

26. Rendering a List in React

```
const students = [  
  "Amit",  
  "Rahul",  
  "Priya"  
];  
  
function App() {  
  
  return (  
    <div>  
  
      {students.map((student) => (  
        <p>{student}</p>  
      ))}  
  
    </div>  
  );  
}
```

27. Keys

React needs a key when rendering lists.

Correct:

```
{students.map((student, index) => (  
  <p key={index}>  
    {student}  
  </p>  
))}
```

However, when your data has a stable unique ID, prefer that:

```
{students.map((student) => (  
  <p key={student.id}>  
    {student.name}  
  </p>  
))}
```

Avoid using array indexes as keys when the list can be reordered, inserted into, or deleted from.

PART 8 — Rendering Objects

28. Student Object Array

```
const students = [  
  {  
    id: 1,  
    name: "Amit",  
    marks: 85,  
    city: "Delhi"  
  },  
  {  
    id: 2,  
    name: "Rahul",  
    marks: 92,  
    city: "Mumbai"  
  },  
  {  
    id: 3,  
    name: "Priya",  
    marks: 78,  
    city: "Pune"  
  }  
];
```

Render:

```
{students.map((student) => (  
  <StudentCard  
    key={student.id}  
    name={student.name}  
    marks={student.marks}  
    city={student.city}  
  />  
))}
```

PART 9 — Controlled Inputs

29. What is a Controlled Input?

In React, a controlled input is an input whose value is managed by React state.

We use:

`useState()`

Example:

```
import { useState } from "react";
```

```
function App() {
```

```
const [name, setName] = useState("");

return (
  <input
    value={name}
    onChange={(e) => setName(e.target.value)}
  />
);
}
```

Flow:

User types



onChange



setName()



State changes



UI updates

PART 10 — Complete Student Form

Now let's create a complete form.

We will include:

- Name
- Email
- Course
- Gender
- Skills
- City
- Form submit

30. StudentForm.jsx

Create:

src/components/StudentForm.jsx

Code:

```
import { useState } from "react";
```

```
function StudentForm() {
```

```
  const [formData, setFormData] = useState({
```

```
name: "",
email: "",
course: "",
gender: "",
city: "",
skills: []
});
```

```
function handleChange(event) {

  const { name, value } = event.target;

  setFormData({
    ...formData,
    [name]: value
  });
}
```

```
function handleSkillChange(event) {

  const { value, checked } = event.target;

  if (checked) {

    setFormData({
      ...formData,
      skills: [...formData.skills, value]
    });

  } else {

    setFormData({
      ...formData,
      skills: formData.skills.filter(
        (skill) => skill !== value
      )
    });

  }
}
```

```
function handleSubmit(event) {  
  
    event.preventDefault();  
  
    console.log("Student Data:", formData);  
  
    alert("Student registered successfully!");  
}
```

```
return (  
    <form  
        className="student-form"  
        onSubmit={handleSubmit}  
    >  
  
        <h2>Student Registration</h2>  
  
        {/* Name */}  
  
        <div className="form-group">  
  
            <label>Name</label>  
  
            <input  
                type="text"  
                name="name"  
                value={formData.name}  
                onChange={handleChange}  
                placeholder="Enter student name"  
            />  
  
        </div>  
  
        {/* Email */}  
  
        <div className="form-group">  
  
            <label>Email</label>  
  
            <input  
                type="email"
```

```
        name="email"
        value={formData.email}
        onChange={handleChange}
        placeholder="Enter email"
      />

</div>

{/* Course */}

<div className="form-group">

  <label>Course</label>

  <select
    name="course"
    value={formData.course}
    onChange={handleChange}
  >

    <option value="">
      Select Course
    </option>

    <option value="BCA">
      BCA
    </option>

    <option value="B.Tech">
      B.Tech
    </option>

    <option value="MCA">
      MCA
    </option>

  </select>

</div>

{/* City */}
```

```
<div className="form-group">
```

```
<label>City</label>
```

```
<select  
  name="city"  
  value={formData.city}  
  onChange={handleChange}  
>
```

```
<option value="">  
  Select City  
</option>
```

```
<option value="Delhi">  
  Delhi  
</option>
```

```
<option value="Mumbai">  
  Mumbai  
</option>
```

```
<option value="Pune">  
  Pune  
</option>
```

```
</select>
```

```
</div>
```

```
{/* Gender */}
```

```
<div className="form-group">
```

```
<label>Gender</label>
```

```
<div className="options">
```

```
<label>  
  <input
```

```
    type="radio"
    name="gender"
    value="Male"
    checked={formData.gender === "Male"}
    onChange={handleChange}
  />
  Male
</label>
```

```
<label>
  <input
    type="radio"
    name="gender"
    value="Female"
    checked={formData.gender === "Female"}
    onChange={handleChange}
  />
  Female
</label>
```

```
</div>
```

```
</div>
```

```
{/* Skills */}
```

```
<div className="form-group">
```

```
<label>Skills</label>
```

```
<div className="options">
```

```
<label>
  <input
    type="checkbox"
    value="Java"
    checked={formData.skills.includes("Java")}
    onChange={handleSkillChange}
  />
  Java
</label>
```

```

    <label>
      <input
        type="checkbox"
        value="React"
        checked={formData.skills.includes("React")}
        onChange={handleSkillChange}
      />
      React
    </label>

    <label>
      <input
        type="checkbox"
        value="Python"
        checked={formData.skills.includes("Python")}
        onChange={handleSkillChange}
      />
      Python
    </label>

  </div>

</div>

<button type="submit">
  Register Student
</button>

</form>
);
}

export default StudentForm;

```

31. CSS for Form

Add this to App.css.

```

.student-form {
  max-width: 550px;
  margin: 40px auto;
  background: white;
}

```

```
padding: 30px;
border-radius: 18px;
box-shadow: 0 10px 30px rgba(0, 0, 0, 0.08);
}
```

```
.student-form h2 {
  text-align: center;
  margin-bottom: 25px;
  color: #1e293b;
}
```

```
.form-group {
  margin-bottom: 20px;
}
```

```
.form-group > label {
  display: block;
  margin-bottom: 8px;
  font-weight: 600;
  color: #334155;
}
```

```
.form-group input[type="text"],
.form-group input[type="email"],
.form-group select {
  width: 100%;
  padding: 12px;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  font-size: 15px;
  outline: none;
}
```

```
.form-group input:focus,
.form-group select:focus {
  border-color: #4f46e5;
}
```

```
.options {
  display: flex;
  gap: 20px;
```

```
    flex-wrap: wrap;
  }

.options label {
  display: flex;
  align-items: center;
  gap: 6px;
  cursor: pointer;
}

.student-form button {
  width: 100%;
  padding: 13px;
  border: none;
  border-radius: 8px;
  background: #4f46e5;
  color: white;
  font-size: 16px;
  font-weight: bold;
  cursor: pointer;
}

.student-form button:hover {
  background: #4338ca;
}
```

32. Use Form in App.jsx

```
import './App.css';
import StudentForm from './components/StudentForm';

function App() {

  return (
    <div className="app">

      <h1>Student Management System</h1>

      <StudentForm />

    </div>
  );
}
```

```
}
```

```
export default App;
```

PART 11 — Understanding the Form

33. useState

```
const [formData, setFormData] = useState({  
  name: "",  
  email: "",  
  course: "",  
  gender: "",  
  city: "",  
  skills: []  
});
```

We keep the entire form data in one object.

34. Handling Text Input

```
<input  
  name="name"  
  value={formData.name}  
  onChange={handleChange}  
>
```

When the user types:

A

Am

Ami

Amit

React updates:

formData.name

35. Understanding [name]

This code:

```
setFormData({  
  ...formData,  
  [name]: value  
});
```

is very important.

Suppose:

name = "email"

value = "amit@gmail.com"

It effectively updates:

email: "amit@gmail.com"
without removing the other properties.

36. Select

HTML:

```
<select>
```

React controlled select:

```
<select  
  name="course"  
  value={formData.course}  
  onChange={handleChange}  
>
```

37. Radio Button

Radio buttons are generally used when the user should select **one option**.

Example:

```
<input  
  type="radio"  
  name="gender"  
  value="Male"  
  checked={formData.gender === "Male"}  
  onChange={handleChange}  
>
```

Both radio buttons use the same:
name="gender"

38. Checkbox

Checkboxes are suitable for multiple selections.

For example:

☒ Java
☒ React
☐ Python

Our state:

skills: []

can contain:

["Java", "React"]

39. Form Submit

```
function handleSubmit(event) {
```

```
event.preventDefault();
```

```
console.log(formData);
```

```
}
```

Why preventDefault()?

Normally the browser submits the HTML form and may reload/navigate.

React applications commonly prevent that default behavior so we can handle submission ourselves.

PART 12 — Parent-to-Child Communication

40. Example

Parent:

```
function App() {
```

```
  const studentName = "Amit";
```

```
  return (
```

```
    <StudentCard name={studentName} />
```

```
  );
```

```
}
```

Child:

```
function StudentCard({ name }) {
```

```
  return (
```

```
    <h2>
```

```
      Welcome {name}
```

```
    </h2>
```

```
  );
```

```
}
```

Flow:

App

|

| name="Amit"

↓

StudentCard

|

↓

Welcome Amit

PART 13 — Child-to-Parent Communication

Although the main topic is parent-to-child communication, interns should understand the reverse pattern too.

React normally passes data downward through props.

To let a child notify the parent, the parent can pass a **function as a prop**.

Parent

```
function App() {  
  
  function showMessage(message) {  
    alert(message);  
  }  
  
  return (  
    <StudentButton  
      onClick={showMessage}  
    />  
  );  
}
```

Child

```
function StudentButton({ onClick }) {  
  
  return (  
    <button  
      onClick={() => onClick("Student button clicked")}  
    >  
      Click Me  
    </button>  
  );  
}
```

Flow:

Parent

↓ function prop

Child

↓ calls function

Parent function executes

This pattern is very important in real React applications.

PART 14 — Complete Mini Project

Now let's combine the concepts.

Our application will have:

Student Management System



Student Form



Student List



Student Cards

41. StudentList.jsx

```
import StudentCard from "../StudentCard";
```

```
function StudentList({ students }) {
```

```
  return (  
    <div className="student-container">
```

```
      {students.map((student) => (
```

```
        <StudentCard  
          key={student.id}  
          name={student.name}  
          marks={student.marks}  
          city={student.city}  
          course={student.course}  
        />
```

```
      )))
```

```
    </div>
```

```
  );
```

```
}
```

```
export default StudentList;
```

42. StudentCard.jsx

```
function StudentCard({
```

```
name,  
marks,  
city,  
course  
}) {  
  
return (  
  <div className="student-card">  
  
    <div className="student-avatar">  
      {name.charAt(0)}  
    </div>  
  
    <div className="student-info">  
  
      <h2>{name}</h2>  
  
      <p>  
        <strong>Course:</strong> {course}  
      </p>  
  
      <p>  
        <strong>City:</strong> {city}  
      </p>  
  
      <p>  
        <strong>Marks:</strong> {marks}  
      </p>  
  
      {marks >= 90 && (  
        <p className="excellent">  
          ★ Excellent Student  
        </p>  
      )}  
  
      <p  
        className={  
          marks >= 40  
            ? "pass"  
            : "fail"  
        }  
      >
```

```
    >
      {marks >= 40 ? "Pass" : "Fail"}
    </p>

  </div>

</div>
);
}
```

```
export default StudentCard;
```

43. App.jsx

```
import './App.css';
import StudentList from './components/StudentList';

function App() {

  const students = [
    {
      id: 1,
      name: "Amit",
      marks: 85,
      city: "Delhi",
      course: "B.Tech"
    },
    {
      id: 2,
      name: "Rahul",
      marks: 92,
      city: "Mumbai",
      course: "BCA"
    },
    {
      id: 3,
      name: "Priya",
      marks: 78,
      city: "Pune",
      course: "MCA"
    },
    {
```

```

      id: 4,
      name: "Neha",
      marks: 95,
      city: "Delhi",
      course: "B.Tech"
    }
  ];

  return (
    <div className="app">

      <h1>Student Management System</h1>

      <p className="subtitle">
        React Props, Events, Lists & Conditional Rendering
      </p>

      <StudentList students={students} />

    </div>
  );
}

export default App;

```

44. Final CSS

```

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
  background: #f1f5f9;
  color: #1e293b;
}

.app {
  min-height: 100vh;
  padding: 50px 25px;
}

```

```
}
```

```
.app h1 {  
  text-align: center;  
  font-size: 36px;  
  margin-bottom: 10px;  
}
```

```
.subtitle {  
  text-align: center;  
  color: #64748b;  
  margin-bottom: 40px;  
}
```

```
.student-container {  
  max-width: 1000px;  
  margin: auto;  
  
  display: grid;  
  grid-template-columns:  
    repeat(auto-fit, minmax(300px, 1fr));  
  
  gap: 25px;  
}
```

```
.student-card {  
  background: white;  
  padding: 25px;  
  border-radius: 18px;  
  
  display: flex;  
  align-items: center;  
  gap: 20px;  
  
  box-shadow:  
    0 10px 25px rgba(0, 0, 0, 0.08);  
  
  transition: transform 0.3s;  
}
```

```
.student-card:hover {
```

```
    transform: translateY(-6px);  
}
```

```
.student-avatar {  
    min-width: 65px;  
    height: 65px;  
  
    border-radius: 50%;  
  
    display: flex;  
    align-items: center;  
    justify-content: center;  
  
    background: #4f46e5;  
    color: white;  
  
    font-size: 25px;  
    font-weight: bold;  
}
```

```
.student-info h2 {  
    margin-bottom: 10px;  
}
```

```
.student-info p {  
    margin: 6px 0;  
    color: #64748b;  
}
```

```
.excellent {  
    color: #d97706 !important;  
    font-weight: bold;  
}
```

```
.pass {  
    color: #16a34a !important;  
    font-weight: bold;  
}
```

```
.fail {  
    color: #dc2626 !important;
```

```
font-weight: bold;
}
```

45. What Concepts Did We Use?

The final project uses:

Props

```
<StudentList students={students} />
```

Multiple Props

```
<StudentCard
  name={student.name}
  marks={student.marks}
  city={student.city}
  course={student.course}
/>
```

Destructuring

```
function StudentCard({
  name,
  marks,
  city,
  course
})
```

Component Reusability

```
<StudentCard />
```

used multiple times through `map()`.

List Rendering

```
students.map(...)
```

Keys

```
key={student.id}
```

Conditional Rendering

```
marks >= 40
```

Ternary

```
marks >= 40 ? "Pass" : "Fail"
```

Logical AND

```
marks >= 90 && (...)
```

PART 15 — Common Mistakes

Mistake 1 — Calling event handler immediately

 Wrong:

```
<button onClick={handleClick()}>
```

 Correct:

```
<button onClick={handleClick}>
```

Mistake 2 — Forgetting key



```
students.map(student => (  
  <StudentCard />  
))
```



```
students.map(student => (  
  <StudentCard key={student.id} />  
))
```

Mistake 3 — Using class

React uses:

className

not:

class

Mistake 4 — Forgetting value in controlled input

```
<input  
  value={name}  
  onChange={handleChange}  
/>
```

Both value and onChange are important when controlling an input with state.

Mistake 5 — Mutating state directly



```
formData.name = "Amit";
```

Instead use the setter:

```
setFormData({  
  ...formData,  
  name: "Amit"  
});
```

PART 16 — Assignments

Assignment 1 — Product Card

Create a reusable:

```
<ProductCard />
```

It should receive:

name

price

category

rating

Display all information.

Solution

```
function ProductCard({
  name,
  price,
  category,
  rating
}) {

  return (
    <div className="student-card">

      <h2>{name}</h2>

      <p>Price: ₹{price}</p>

      <p>Category: {category}</p>

      <p>Rating: ★ {rating}</p>

    </div>
  );
}
```

export default ProductCard;

Use:

```
<ProductCard
  name="Laptop"
  price={55000}
  category="Electronics"
  rating={4.5}
/>
```

Assignment 2 — Pass/Fail

Create a component that receives marks.

If marks are 40 or above:

PASS

Otherwise:

FAIL

Solution

```
function Result({ marks }) {

  return (
    <h2>
      {marks >= 40 ? "PASS" : "FAIL"}
    </h2>
  );

}

export default Result;
```

Assignment 3 — List of Products

Create:

```
const products = [
  {
    id: 1,
    name: "Laptop",
    price: 55000
  },
  {
    id: 2,
    name: "Phone",
    price: 25000
  },
  {
    id: 3,
    name: "Headphones",
    price: 3000
  }
];
```

Display them using map().

Solution

```
function App() {

  const products = [
    {
      id: 1,
      name: "Laptop",
      price: 55000
```

```

    },
    {
      id: 2,
      name: "Phone",
      price: 25000
    },
    {
      id: 3,
      name: "Headphones",
      price: 3000
    }
  ];

  return (
    <div>

      {products.map((product) => (

        <div key={product.id}>

          <h2>{product.name}</h2>

          <p>
            ₹{product.price}
          </p>

          </div>

        )}}

    </div>
  );
}

```

Assignment 4 — Login Form

Create a form containing:

Email

Password

Login button

Use controlled inputs.

Solution

```
import { useState } from "react";

function LoginForm() {

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  function handleSubmit(event) {

    event.preventDefault();

    console.log({
      email,
      password
    });

  }

  return (
    <form onSubmit={handleSubmit}>

      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />

      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder="Password"
      />

      <button type="submit">
        Login
      </button>

    </form>
  );
}
```

```
}
```

```
export default LoginForm;
```

Assignment 5 — Skills Form

Create checkboxes:

☐ Java

☐ Python

☐ React

☐ Node.js

Allow the user to select multiple skills and display selected skills after submission.

Solution

```
import { useState } from "react";
```

```
function SkillsForm() {
```

```
  const [skills, setSkills] = useState([]);
```

```
  function handleChange(event) {
```

```
    const {
      value,
      checked
    } = event.target;
```

```
    if (checked) {
```

```
      setSkills([
        ...skills,
        value
      ]);
```

```
    } else {
```

```
      setSkills(
        skills.filter(
          skill => skill !== value
        )
      );
```

```

    }
  }

  return (
    <div>

      <h2>Select Skills</h2>

      {[ "Java", "Python", "React", "Node.js" ].map(
        skill => (

          <label key={skill}>
            <input
              type="checkbox"
              value={skill}
              checked={skills.includes(skill)}
              onChange={handleChange}
            />

            {skill}
          </label>

        )
      )}

      <h3>Selected Skills:</h3>

      <p>
        {skills.join(", ")}
      </p>

    </div>
  );
}

```

```
export default SkillsForm;
```

Assignment 6 — Intern Challenge

Create a **Student Registration Dashboard**.

It should contain:

Student Form

Name

Email

Course

City

Gender

Skills

Submit

Student List

After submitting:

Student Name

Email

Course

City

Gender

Skills

should appear as a card.

Requirements

Students must use:

- useState
- Props
- Parent → Child communication
- Child → Parent communication using a callback prop
- map()
- key
- Controlled inputs
- Select
- Radio
- Checkbox
- Form submit
- Conditional rendering
- CSS

Expected architecture

App

|

|— StudentForm

|

| |
| |— onAddStudent()

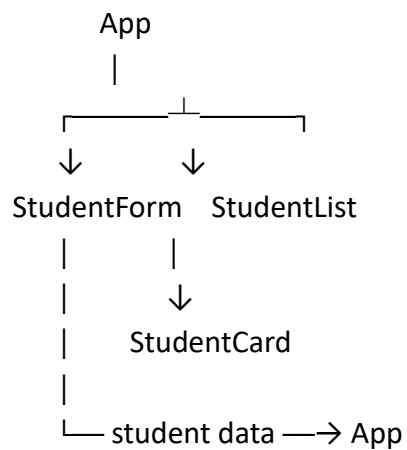
|

|— StudentList

|

|— StudentCard

Data flow:



This assignment is excellent practice because it combines almost every concept from this lesson.

Quick Revision Sheet

Props

```
<Component name="Amit" />
```

Receive:

```
function Component({ name }) {
```

Multiple Props

```
<Student
  name="Amit"
  age={22}
  marks={85}
/>
```

Event

```
<button onClick={handleClick}>
```

Input Event

```
<input
  value={name}
  onChange={(e) => setName(e.target.value)}
/>
```

Ternary

```
condition ? "Yes" : "No"
```

Logical AND

```
condition && <Component />
```

List

```
items.map(item => (  
  <Component key={item.id} />  
))
```

Controlled Input

```
const [name, setName] = useState("");
```

```
<input  
  value={name}  
  onChange={(e) => setName(e.target.value)}  
/>
```

Select

```
<select  
  value={course}  
  onChange={handleChange}  
>
```

Radio

```
<input  
  type="radio"  
  name="gender"  
  value="Male"  
/>
```

Checkbox

```
<input  
  type="checkbox"  
  value="React"  
/>
```

Form

```
<form onSubmit={handleSubmit}>  
and:  
function handleSubmit(e) {  
  e.preventDefault();  
}
```

Final Learning Flow

Components



Props



Multiple Props



Destructuring



Reusable Components



Component Composition



Events



Event Parameters



Conditional Rendering



Ternary + &&



Arrays + map()



Keys



Objects in Lists



Controlled Inputs



Forms



Select



Radio



Checkbox



Parent → Child



Child → Parent via Callback



Mini Student Management Project

Important intern takeaway: React applications become much easier to understand if students keep one mental model in mind: **data normally flows down through props, user actions trigger events, state changes update the UI, and map() turns data arrays into reusable components.**